

Predictive_Analytics_Project_3

February 12, 2026

Problem to demonstrate the role of qualitative (nominal) predictors in addition to quantitative predictors in multiple linear regression: Attach “Credits” data from R. Regress “balance” on: 1. “gender” only. 2. “gender” and “ethnicity” 3. “gender”, “ethnicity”, “income”. 4. Output all the regressions in (1) - (3) in a single table using stargazer. Comment on the significant coefficients in each of the models. 5. Explain how gender affects “balance” in each of the models (1) - (3) . 6. Compare the average credit card balance of a male African with a male Caucasian on the basis of model (2). 7. Compare the average credit card balance of a male African with a male Caucasian when each earns 100,000 dollars. For comparison, use the model in (3). 8. Compare and comment on the answers in (6) and (7) 9. Based on the model in (3), predict the credit card balance of a female Asian whose income is 2000,000 dollars. 10. Check the goodness of fit of the different models in (1)-(3) in terms of AIC, BIC and adjusted R2. Which model would you prefer?

```
[1]: import pandas as pd
import numpy as np
import statsmodels.formula.api as smf
from statsmodels.iolib.summary2 import summary_col
```

```
[4]: from ISLP import load_data
Credit = load_data('Credit')
df = pd.DataFrame(Credit)
df.head()
```

```
[4]:
```

	ID	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	\
0	1	14.891	3606	283	2	34	11	Male	No	Yes	
1	2	106.025	6645	483	3	82	15	Female	Yes	Yes	
2	3	104.593	7075	514	4	71	11	Male	No	No	
3	4	148.924	9504	681	3	36	11	Female	No	No	
4	5	55.882	4897	357	2	68	16	Male	No	Yes	

	Ethnicity	Balance
0	Caucasian	333
1	Asian	903
2	Asian	580
3	Asian	964
4	Caucasian	331

```
[5]: fit1 = smf.ols('Balance ~ Gender', data=df).fit()
fit2 = smf.ols('Balance ~ Gender + Ethnicity', data=df).fit()
```

```
fit3 = smf.ols('Balance ~ Gender + Ethnicity + Income', data=df).fit()
```

```
[7]: print(summary_col([fit1, fit2, fit3],
    stars=True,
    model_names=['Model 1', 'Model 2', 'Model 3'],
    info_dict={'N':lambda x: "{0:d}".format(int(x.nobs)),
    'Adj. R2':lambda x: "{:.3f}".format(x.rsquared_adj)}))
```

```
=====
```

	Model 1	Model 2	Model 3
Intercept	509.8031*** (33.1281)	520.8797*** (51.9012)	230.0291*** (53.8574)
Gender[T.Female]	19.7331 (46.0512)	20.0382 (46.1778)	24.3396 (40.9630)
Ethnicity[T.Asian]		-19.3709 (65.1068)	1.6372 (57.7867)
Ethnicity[T.Caucasian]		-12.6530 (56.7401)	6.4469 (50.3634)
Income			6.0542*** (0.5818)
R-squared	0.0005	0.0007	0.2157
R-squared Adj.	-0.0021	-0.0069	0.2078
Adj. R2	-0.002	-0.007	0.208
N	400	400	400

```
=====
Standard errors in parentheses.
* p<.1, ** p<.05, ***p<.01
```

```
[12]: fit1.pvalues.get('Gender[T.Female]', fit2.pvalues.get('Gender[T.Male]'))
```

```
[12]: np.float64(0.6685161055027202)
```

In the light of our calculations, we see that the p-value is greater than 0.05 indicating that there is no statistical evidence that Gender affects Balance.

```
[14]: params_1 = fit1.params
    params_2 = fit2.params
    params_3 = fit3.params
```

```
[16]: diff_2 = params_2.get('Ethnicity[T.Caucasian]', 0) - 0
    diff_3 = params_3.get('Ethnicity[T.Caucasian]', 0) - 0
    print(f"Estimated difference (Caucasian - African American): {diff_2:.2f}")
    print(f"Estimated difference controlling for Income: {diff_3:.2f}")
```

```
Estimated difference (Caucasian - African American): -12.65
Estimated difference controlling for Income: 6.45
```

Thus, there is no statistical evidence of a difference in credit card balance between African Americans and Caucasians. The numbers -12.65 and 6.45 are likely just statistical noise around zero.

```
[18]: new_data = pd.DataFrame({
        'Gender': ['Female'],
        'Ethnicity': ['Asian'],
        'Income': [2000]})
predicted_balance = fit3.predict(new_data)
print(f"Predicted Balance: ${predicted_balance.iloc[0]:.2f}")
```

Predicted Balance: \$12364.46

```
[20]: models = {'Model A': fit1, 'Model B': fit2, 'Model C': fit3}
metrics = pd.DataFrame({
        'AIC': [m.aic for m in models.values()],
        'BIC': [m.bic for m in models.values()],
        'Adj R^2': [m.rsquared_adj for m in models.values()]
    }, index=models.keys())
print(metrics)
```

	AIC	BIC	Adj R ²
Model A	6042.526817	6050.509746	-0.002050
Model B	6046.433622	6062.399480	-0.006877
Model C	5951.517634	5971.474957	0.207774

Problem to demonstrate the impact of ignoring interaction term in multiple linear regression Consider a simulation setting where the data is generated as follows: 1. Generate x_{1i} from $Normal(0,1)$ distribution, $i = 1, 2, \dots, n$ 2. Generate x_{2i} from $Bernoulli(0.3)$ distribution, $i = 1, 2, \dots, n$ 3. Generate ϵ_i from $Normal(0,1)$ and hence generate the response $y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \beta_3 (x_{1i} * x_{2i}) + \epsilon_i, i = 1, 2, \dots, n$. 4. Run two separate multiple linear regressions (i) using the model in Step 3 and (ii) using the model in Step 3 without the interaction term. Repeat Steps 1-4, $R = 1000$ times. At each simulation compute the MSE for the correct model (i.e. model with the interaction term) and the naive model (i.e. the model without the interaction term). Finally find the average MSE's for each model. From the output, demonstrate the impact of ignoring the interaction term. Carry out the analysis for $n = 100$ and the following parametric configurations: $(\beta_0, \beta_1, \beta_2, \beta_3) = (-2.5, 1.2, 2.3, 0.001), (-2.5, 1.2, 2.3, 3.1)$. Set seed as 123.

```
[21]: from sklearn.metrics import mean_squared_error
def run_simulation(n_simulations, n_samples, beta_config, seed=123):
    np.random.seed(seed)
    beta0, beta1, beta2, beta3 = beta_config

    mse_interaction = []
    mse_naive = []
    for _ in range(n_simulations):
        x1 = np.random.normal(0, 1, n_samples)
        x2 = np.random.binomial(1, 0.3, n_samples)
        epsilon = np.random.normal(0, 1, n_samples)
        y = beta0 + beta1*x1 + beta2*x2 + beta3*(x1*x2) + epsilon
```

```

df = pd.DataFrame({'y': y, 'x1': x1, 'x2': x2})
model_true = smf.ols('y ~ x1 * x2', data=df).fit()
preds_true = model_true.predict(df)
mse_interaction.append(mean_squared_error(y, preds_true))
model_naive = smf.ols('y ~ x1 + x2', data=df).fit()
preds_naive = model_naive.predict(df)
mse_naive.append(mean_squared_error(y, preds_naive))

return np.mean(mse_interaction), np.mean(mse_naive)

```

```

[22]: n = 100
simulations = 1000
config_1 = (-2.5, 1.2, 2.3, 0.001)
config_2 = (-2.5, 1.2, 2.3, 3.1)

```

```

[23]: mse_true_1, mse_naive_1 = run_simulation(simulations, n, config_1)
print(f"Config 1 (Small Interaction): MSE with Interaction = {mse_true_1:.4f},  

      ↪MSE without Interaction = {mse_naive_1:.4f}")

```

Config 1 (Small Interaction): MSE with Interaction = 0.9615, MSE without Interaction = 0.9714

Hence, with a negligible interaction term, the naive model performs similarly to the true model.

```

[24]: mse_true_2, mse_naive_2 = run_simulation(simulations, n, config_2)
print(f"Config 2 (Large Interaction): MSE with Interaction = {mse_true_2:.4f},  

      ↪MSE without Interaction = {mse_naive_2:.4f}")

```

Config 2 (Large Interaction): MSE with Interaction = 0.9615, MSE without Interaction = 2.8369

Based on the calculations, we can see that with a large interaction term, ignoring it (Naive Model) significantly increases MSE.